

A serious LakeHouse on a seriously tight budget

Bronze → Silver → Gold with open data and free tools

Danish democracy, RNA families, and one analytical database doing all the work

Why this talk exists

- I wanted a data platform I could run **on a laptop, for €0** — but that still does the grown-up things: CDC, history, orchestration, backups.
- Constraint forces good decisions. No Spark cluster to hide behind.
- Everything here is open source. The *only* optional paid piece is the cloud export target.
- It's a **learning project** — so I get to show you the scars, not just the architecture diagram.

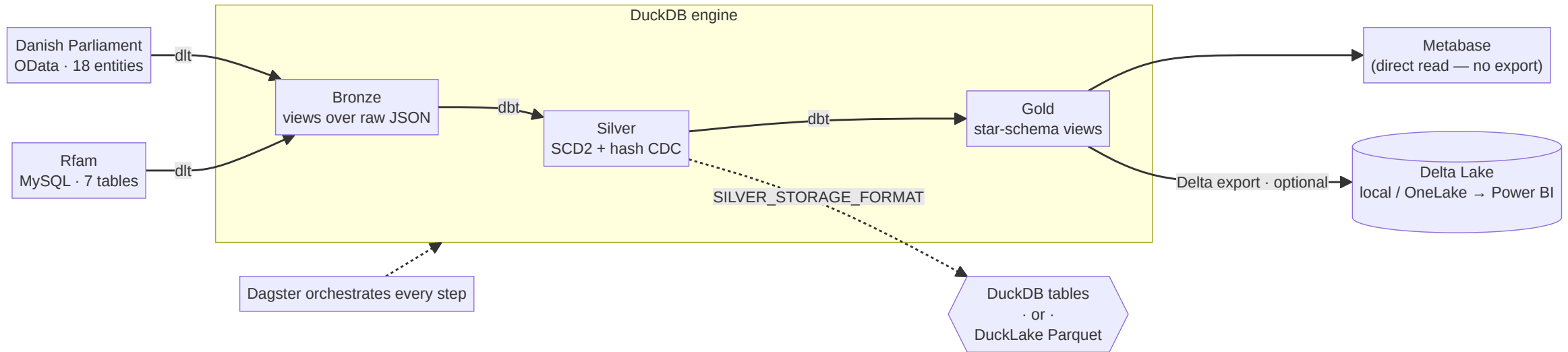
The odd-couple dataset

Two public sources, deliberately mismatched:

	Danish Parliament (Folketing)	Rfam (EBI)
What	MPs, meetings, cases, votes	RNA families, genomes, clans
Where	<code>oda.ft.dk</code> OData REST API	public MySQL at EBI
Size	18 entities	7 tables
Why	rich relational graph, real history	a <i>completely</i> different shape

- 25 entities total. No API key. No rate limit drama. No NDA.
- The mismatch is the point: if the pipeline only works for one source's shape, it's not a pipeline, it's a script.

The stack, on one slide



- **Extract:** dlt · **Transform:** dbt + DuckDB · **Orchestrate:** Dagster · **Export:** deltalake · **BI:** Metabase (direct read).
- New to some here: **dlt** = lightweight Python extract-load; **Dagster** = the orchestrator — think SQL Agent / ADF, but code-first.



dltHub



Medallion, the honest version

- **Bronze** — raw, untouched, queryable. JSON exactly as it landed.
- **Silver** — **fully historised** (SCD Type 2). Barely cleaned, *not* enriched — and that's deliberate.
- **Gold** — business-friendly. English names, surrogate keys, star schema.

Let me be honest about Silver: it doesn't scrub your data and it doesn't enrich it. Its **one job** is to capture **what changed and when**, losslessly. That's still where the engineering lives — hash-based CDC, SCD Type 2, and a load you can safely re-run — just don't mistake it for a cleaning step.

Extraction with dlt

- Two orchestrators, one engine: `api_to_file()` for OData, `sql_to_file()` for MySQL.
- Each source declares **incremental** vs **full-extract** per entity:
 - DDD: 6 incremental (filtered on `opdateringsdato`), 12 full lookups.
 - Rfam: 2 incremental (`updated`), 5 full.
- Incremental state lives in dlt; output is **timestamped JSON** (`{entity}_{YYYYMMDD_HHMMSS}.json`).
- `ThreadPoolExecutor(max_workers=4)` — extraction is I/O-bound, so threads are free lunch.

Bronze: the easy layer

- 53 views, all generated. Most are literally `read_json_auto()` over a folder.
- A `_latest` view per entity (newest snapshot) and a few utility views.
- Zero copies. Bronze is a **lens on files**, not a table.

```
-- essentially this, 25 times
SELECT *, '<filename>' AS LKHS_filename
FROM read_json_auto('.../Bronze/ddd/aktoer/*.json')
```

Silver, part 1: detecting change with a hash

One idea: **hash each row, compare it to the same key's previous snapshot**. No `updated_at` to trust, no column-by-column diff.

- SHA-256 over the *business* columns only — exclude bookkeeping (insert time, filename, run id) or every row looks changed every run.

```
with hashed as (                                -- 1) hash the business columns
    select id, LKHS_filename,
           sha256(concat_ws('|', navn, parti, rolle)) as hash
    from   bronze_ddd_aktoer
),
lagged as (                                     -- 2) each key's previous snapshot
    select *, lag(hash) over (partition by id order by LKHS_filename) as prev
    from   hashed
)
select *,
       case when prev is null then 'I'          -- new key          -> insert
            when hash <> prev then 'U'          -- hash changed -> update
            end as LKHS_cdc_operation           -- (deletes: a separate pass)
from   lagged;
```

- **Order- and batch-independent:** the `LAG()` runs over the *whole* file history, so processing five new extractions at once yields **exactly** the same rows as one-by-one. Idempotent and safe to re-run.

Silver, part 2: SCD Type 2 + current-version views

- Every change is **appended**, never overwritten. Full history stays in Silver.
- Tracking columns, all prefixed `LKHS_`:
 - `LKHS_date_valid_from` · `LKHS_cdc_operation` (I/U/D) · `LKHS_hash_value`
- A `_cv` ("current version") view per table exposes just the latest row — so downstream consumers don't have to think about history unless they want to.

History for the auditors, `_cv` for everyone else. Same table.

The hash-based CDC + SCD Type 2 pattern here is inspired by **Roelant Vos**'s work on data-warehouse automation and historisation.

Silver, part 3: I don't write Silver SQL

- Bronze and Silver models are **generated** from one Python file (`configuration_variables.py`).
- That file is the single source of truth: entity lists, primary keys, date columns, SQL templates.
- Add an entity → edit *one* list → regenerate. The macros do the CDC.
- Tests assert the lists stay consistent (counts, subsets, no dupes), so a typo fails CI, not production.

```
# add a Rfam table = append to four lists, then:  
python -m ddd_python.ddd_dbt.generate_dbt_models
```

Gold: making it presentable

- 19 views: 10 star-schema facts/dims + `_cv` passthroughs + a date dimension.
- English names (`bronze_ddd_aktoer` → `gold_actor`), surrogate keys.
- Surrogate keys cast **UBIGINT** → **signed BIGINT** — because Power BI can't digest unsigned 64-bit ints. (You learn this the annoying way.)
- Mostly generated; a couple handcrafted where the star schema needed a human (`individual_votes`).

Orchestration with Dagster

- Software-defined **assets**, created by factory functions (25 extraction assets from ~2 factories).
- Jobs: incremental, full-extract, full-pipeline, export, DuckLake cleanup.
- Two daily schedules (06:00 + 08:00, Europe/Copenhagen) — **shipped disabled**, because a demo that hammers a public API on a cron is rude.
- Run-status sensors log per-run summaries.

One constraint to account for: single writer

DuckDB is **single-writer** — one process holds the write lock on the file. It didn't reshape the architecture; it's just a fact you have to *realise and account for*. Once you do, it's a few small choices:

- **dbt jobs** → `in_process_executor` . Everything that writes the DB runs in one process, serialized. No two models race for the lock.
- **Extraction & export** → `multiprocess_executor` (**max 4**). These write *files*, not the DB, so they're free to run in parallel.
- **Metabase holds a read lock** → **Dagster stops it before a run, starts it after**. The pipeline owns the file for the duration of a build.
- **Human connections (DBeaver, etc.) release during a run** — same rule, applied to people.

Recognise it, arrange *who* runs *where*, move on. Boring, and it never deadlocks.

Silver storage, switchable: DuckDB vs DuckLake

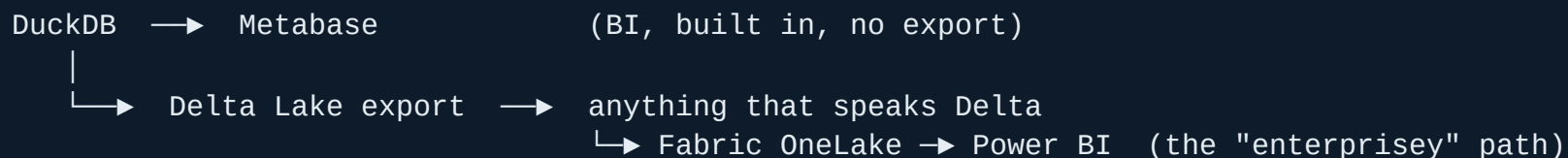
One env var — `SILVER_STORAGE_FORMAT` — flips Silver between two backends:

	duckdb (default)	ducklake
Silver lives in	the <code>.duckdb</code> file	DuckLake catalog (Parquet + metadata)
Files on disk	inside the binary	Parquet under <code>DUCKLAKE_DATA_PATH</code>
Time travel	no	yes (catalog snapshots)

- Bronze and Gold don't care — Gold's `ref()` to Silver just resolves to the catalog.
- Catching detail: DuckLake stores *tiny* tables inline in the catalog, so not every table shows up as a `.parquet` file. First time, you'll think it's broken. It isn't.

The Delta export — and who it's actually for

Not Metabase. Metabase reads the DuckDB file directly — dashboards work with **zero** export. The Delta export is an **optional outbound adapter** for *other* tooling:



- Silver → Delta (incremental append), Gold → Delta (full overwrite); target flips **local** ↔ **OneLake** with one env var.
- Dedup read runs *inside* DuckDB via `delta_scan()`; the **write** still uses the `delta lake` lib. Folding the write into DuckDB is blocked by an **Azure OneLake bug** — upstream, not DuckDB's fault — so for now we wait it out.
- Local-only? You may never run it. Want Power BI / Spark / Databricks? Now Delta on OneLake earns its keep.

My favourite trick: the pipeline watches itself

- Dagster stores its run history in a SQLite database.
- So... point dbt at that SQLite file too.
- A small **data-engineering** model layer turns Dagster's own runs into queryable tables — surfaced in Metabase like any other data.

The pipeline reports on the pipeline. No extra observability stack, just one more dbt source.

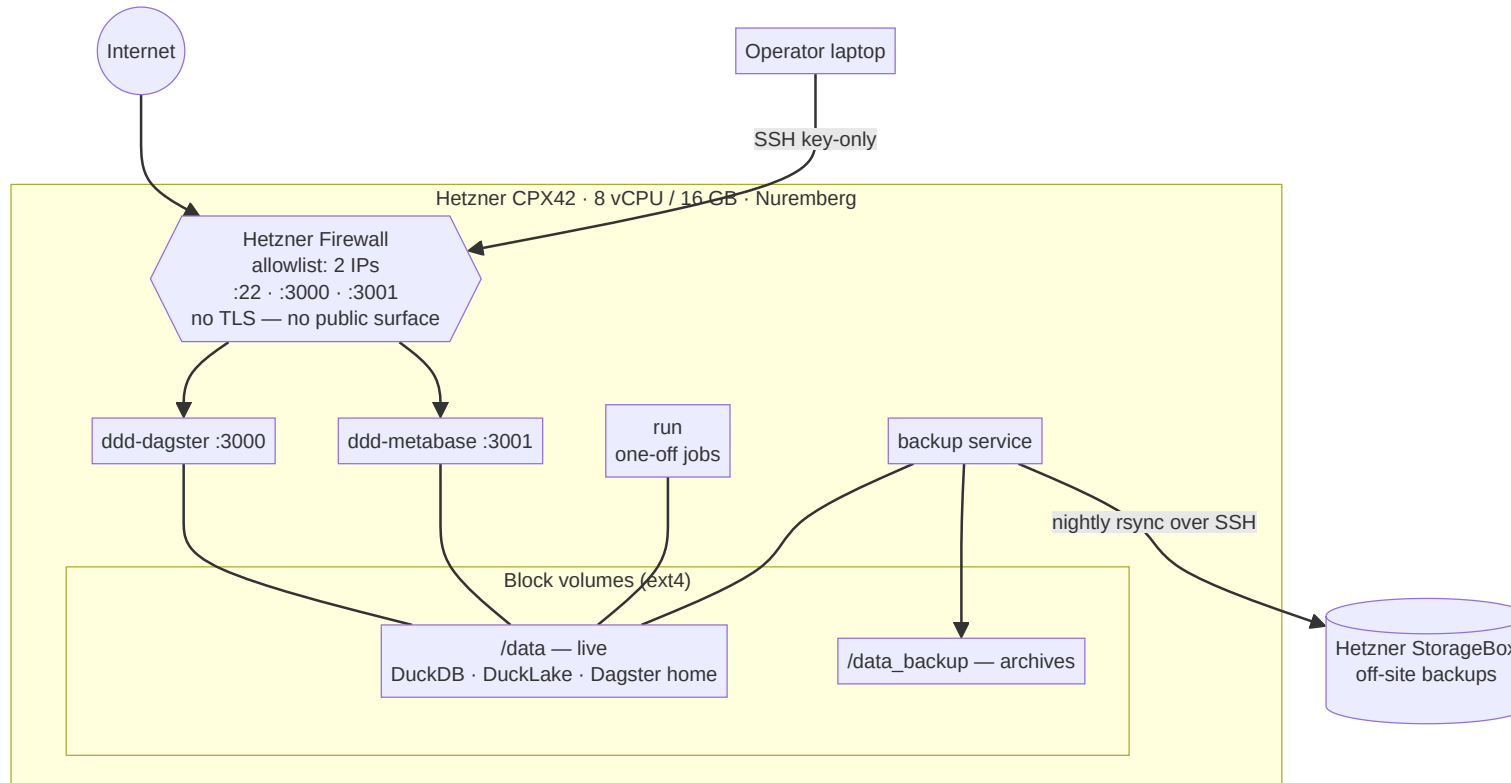
Backups, and a genuinely fun ordering puzzle

Four targets: `dagster`, `metabase`, `duckdb`, and (in DuckLake mode) `ducklake`.

- Retention: 62 days for Dagster/Metabase, **7 days for duckdb/ducklake**.
- The puzzle: the DuckLake **catalog** lives with the `.duckdb` file; the **data files** live elsewhere. The catalog *references* the files.
- So you must archive **data files first, catalog second** — otherwise the catalog snapshot can point at a Parquet file the backup never captured.

Files-before-catalog. Correct by construction.

How it's wired in production



Non-root containers (UID 1000 / 2000) · `restart: unless-stopped` · one node, no HA.

Runs on a laptop — deployed on one small box

Dev on a laptop. Production is **one Hetzner box**, run like a grown-up:

- **Host:** CPX42 (8 vCPU / 16 GB), Docker CE, Nuremberg — **~€30/mo**. Two `fstab` volumes: `/data` (live) + `/data_backup`.
- **Security is a whitelist, not a fortress:** firewall allows inbound from exactly **two IPs** (home + VPN exit); ports 22/3000/3001, everything else dropped. **No TLS — on purpose:** no public surface, the allowlist *is* the perimeter. Key-only root SSH.
- **Off-site backups** nightly via cron → Hetzner StorageBox. **Non-root containers** (UID 1000 / 2000), memory limits, `restart: unless-stopped`.

All-in cost: local-only = **€0** (+ ~€30/mo if you host it on Hetzner). Fabric OneLake adds ~€65/mo paused-overnight, ~€190 always-on — *only* if you want the cloud export.

Deliberately *not* built: real-time (batch/daily), alerting (just logged summaries), HA/failover (one node, on purpose), dev/staging/prod split. A reference project, not a platform.

What I'd tell past me

- **Generate** repetitive models; hand-write the interesting 10%.
- A **content hash** beats trusting someone else's `updated_at` .
- Keep **history cheap to ignore** (`_cv` views) and cheap to keep (append-only).
- Know your engine's quirks (e.g. single-writer) and account for them — they're constraints, not catastrophes.
- Keep what *works* in place, note *why* — sometimes the blocker is upstream, and the fix is patience.

Thanks — questions?

- It's a learning project: open source, runs on a laptop, scars included.
- Two odd datasets, one medallion, single-writer kept honest by the orchestration, and a backup that knows about ordering.
- Repo + docs: `dbt_duckdb_demo` (`README.md` , `documentation/`).

Am I doing anything unique here? Not at all. But it's genuinely fun to build — and I honestly believe plenty of clients would be more than well served by *this*, instead of a far costlier alternative.

Ask me the hard ones.